Towards Resilient Immersive Media Delivery in 6G: Churn- and Failure-Aware GNN Multicast Routing, Validated on a 5G SA Testbed

A

report submitted in partial fulfilment for the award of the degree of

IPG Masters of Technology

in

Information Technology

By

Aaditya Karki : 2022IMT-001

Under the Supervision of

Prof. Aditya Trivedi

Department of Information Technology



ABV-INDIAN INSTITUTE OF INFORMATION TECHNOLOGY AND MANAGEMENT GWALIOR

GWALIOR - 474015, INDIA

DECLARATION

I hereby certify that the work presented in this report, entitled **Towards Resilient Immersive Media Delivery in 6G: Churn- and Failure-Aware GNN Multicast Routing, Validated on a 5G SA Testbed**, in fulfillment of the requirement for the award of the degree of Integrated Post Graduate in Masters of Technology and submitted to the institution is an authentic record of my own work carried out during the period July-2026 to September-2026 under the supervision of **Prof. Aditya Trivedi**. I also cited the reference about the text(s)/figure(s)/table(s) from where they have been taken.

Dated: **Signature of the candidate**

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Dated: **Signature of supervisor**

Acknowledgements

I am deeply grateful to Prof. Aditya Trivedi for his guidance and mentorship throughout this Master's Thesis Project (MTP). His questions about what a learned method must prove before it is trusted, and his encouragement to test ideas on real equipment rather than in simulation alone, shaped the direction of this work. I thank the Department of Information Technology, for access to the 5G Use Case lab. I also thank the authors of the base paper for releasing their code publicly, which made the replication reported here possible. I am grateful to this institution for the opportunity to pursue this MTP and for the resources and facilities it has provided, and to my professors for their mentoring and fair evaluation of our work. Finally, I thank my family and friends for their constant support.

Aaditya K.

Abstract

Multi-quality live video for sixth-generation (6G) immersive services needs multicast trees that stay cheap and stable while viewers join, leave and change quality, and while links fail. The base paper (Wang et al.*, IEEE Transactions on Mobile Computing, 2026) builds near-optimal multi-rate multicast trees with a graph-attention policy network (GPN), but only once, for a fixed audience, and only in simulation. This thesis adds event-driven tree maintenance on top of it: repair operators with correct upgrade and downgrade cost accounting; a learned keep/repair/re-plan policy whose installed tree stays within* $(1 + \varepsilon)$ *of a capacity-aware layered MPH (minimum-cost path heuristic) tree; and OpenFlow fast-failover backup branches applied through two-phase rule updates. Evaluation runs in three tiers: simulation on 50–500-node topologies, Mininet/Ryu emulation, and the DoT 5G Use Case Lab at ABV-IIITM Gwalior with real CPEs and XR headsets.*

This mid-term report covers the objectives, the literature and research gap, the methodology and the replication of the base paper. The replication reproduces the paper's GPN at 1.061× optimal on 30-node graphs (paper: 1.062) and 1.074× on 50-node graphs (paper: 1.070). It also shows that a layered MPH heuristic, absent from the paper's comparison, is at least as cheap as GPN on 77–85% of instances and about 45–70× faster. The GNN's role in this thesis is therefore the keep/repair/re-plan decision under churn, not one-shot tree construction.

Keywords: *6G, Multicast Routing, Graph Neural Networks, Software-Defined Networking, Steiner Tree, Multi-Rate Video, Network Resilience, Fast Failover, 5G Testbed*

Contents

List of Figures vii

List of Tables viii

List of Acronyms ix

1 Introduction 1

1.1 Background and Motivation 2

1.2 Objectives 2

1.3 Organisation of the Report 3

2 Literature Review 4

2.1 Background 5

2.2 Key Related Research 5

2.3 Research Gaps 9

2.4 Problem Formulation 10

3 Proposed Methodology 12

3.1 Problem Formulation 13

3.2 Repair Operators 14

3.3 Learned Decision Policy and MPH Safety Net 14

3.4 Proactive Protection and Safe Updates 16

3.5 Three-Tier Evaluation 16

4 Results and Discussion 18

4.1 Replication Setup 19

4.2 Training and Reproduction 19

4.3 Comparison with Classical Baselines 21
4.4 Discussion 22
4.5 Conclusions 23
4.6 Future Scope 24

Bibliography **25**

List of Figures

2.1 Routing example of the base paper (Lemma 1) 6
2.2 Architecture of the base paper's graph policy network (GPN) 6
2.3 Sequential routing in the base paper: Dijkstra vs. GPN 7
2.4 Base paper results: cost vs. users and the dynamic-addition test 7

3.1 Event-driven tree maintenance loop 13
3.2 Tier 2–3 testbed architecture 17

4.1 Replication pipeline 19
4.2 Validation cost during training (two seeds) 20
4.3 Cost above optimum vs. time per tree (50-node graphs) 22
4.4 Per-tree difference between GPN and MPH costs 22

List of Tables

1.1 Objectives and deliverables 3

2.1 Comparison of the related works 9

3.1 Repair operators 14

3.2 Three-tier evaluation plan 16

4.1 Replication setup compared with the base paper 20

4.2 Reproduction of the base paper's headline numbers 21

4.3 Mean tree cost over 100 graphs per setting 21

4.4 Effect of installing the cheaper of the GPN and MPH trees ($\varepsilon = 0$, no capacities; ranges span the two seeds) 23

List of Acronyms

3GPP	3rd Generation Partnership Project
5G	Fifth Generation
6G	Sixth Generation
BCO	Bee Colony Optimization
CPE	Customer Premises Equipment
DHRL	Deep Hierarchical Reinforcement Learning
DoT	Department of Telecommunications
DP	Dynamic Programming
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
ER	Erdős–Rényi (random graph)
GA	Genetic Algorithm
GAT	Graph Attention Network
GNN	Graph Neural Network
GPN	Graph Policy Network
GPU	Graphics Processing Unit
ILP	Integer Linear Programming
IMT	International Mobile Telecommunications
ITU	International Telecommunication Union
KMB	Kou–Markowsky–Berman heuristic
LR	Learning Rate

LSTM	Long Short-Term Memory
MBS	Multicast–Broadcast Services
MEC	Multi-access Edge Computing
MPH	Minimum-cost Path Heuristic
OVS	Open vSwitch
PDU	Protocol Data Unit
PPO	Proximal Policy Optimization
QoE	Quality of Experience
RAN	Radio Access Network
RST	Recover-aware Steiner Tree
SA	Standalone (5G core)
SDN	Software-Defined Networking
UE	User Equipment
UPF	User Plane Function
VLAN	Virtual Local Area Network
XR	Extended Reality

1

Introduction

This chapter introduces the thesis. Section 1.1 describes live immersive video delivery in 6G networks and the four observations that motivate maintaining multicast trees rather than building them once. Section 1.2 sets out the five objectives of the thesis and the deliverable each one produces. Section 1.3 outlines the remaining chapters of the report.

1.1 Background and Motivation

Live immersive video needs multicast trees that stay correct, cheap and stable while viewers join, leave and change quality, and while links fail. When receivers ask for different qualities, as an XR headset and a phone in the same session do, each tree link must carry the highest quality needed below it, so building and maintaining such a multi-rate tree at minimum cost is a dynamic Steiner tree problem. The base paper of this thesis, Wang *et al.* [1], builds multi-rate multicast trees close to optimal cost with a graph-attention policy network (GPN), but only once, for a fixed audience, and only in simulation. Four observations motivate turning it into a resilient tree-maintenance system:

(i) **6G usage scenarios.** ITU-R IMT-2030 [2] names Immersive Communication as a usage scenario and resilience as a cross-cutting design principle.

(ii) **Duplicate unicast today.** 3GPP Release 15/16 systems have no built-in multicast; 5G Multicast–Broadcast Services arrived only in Release 17 [3]. Group video therefore leaves the core as one unicast stream per viewer.

(iii) **Learned routing is static.** The GPN offers no repair short of a full re-plan, although live sessions change constantly.

(iv) **Testbed opportunity.** The DoT 5G Use Case Lab at ABV-IIITM Gwalior [4] offers a 5G SA core, a gNB, a MEC server, CPEs and XR headsets, so routing decisions can be measured at real user devices.

1.2 Objectives

The thesis keeps the base paper's GNN as the tree builder and wraps it in an event-driven maintenance loop with a classical cost guarantee and fast failure recovery. Its five objectives are listed in Table 1.1 together with the deliverable each one produces.

Table 1.1: Objectives and deliverables

Objective	**Deliverable**
Formulate event-driven multi-rate tree maintenance: tree cost + $\mu\times$ rule changes, under link capacity, with correct upgrade/downgrade accounting	Formal problem and cost model
Implement repair operators (graft, prune-up, layer upgrade/downgrade, re-attach) with an MPH fallback in a Ryu controller module	Simulator, operator library, cost-vs-churn results
Extend the base paper's encoder into a masked keep/repair/re-plan policy	Trained policy, ablation vs. threshold rules
Pre-install backup branches as OpenFlow fast-failover groups; apply changes through two-phase updates	Recovery-time and switch-state measurements
Evaluate in three tiers: simulation, Mininet emulation, 5G lab	Results, live demo, paper draft

1.3 Organisation of the Report

Chapter 2 reviews the literature and states the research gap, problem and research questions. Chapter 3 describes the proposed methodology. Chapter 4 reports the replication of the base paper, discusses its consequences and concludes with the plan for the remaining work.

2

Literature Review

This chapter reviews the literature on which the thesis builds. Section 2.1 covers the background: multicast Steiner trees, the MPH and KMB heuristics and their bounds, multi-rate (layered) delivery and software-defined networking. Section 2.2 discusses the key related works, namely the base paper, learned SDN multicast routers, dynamic Steiner trees and SDN resilience and update mechanisms, each with its limitation, and ends with a comparison table. Section 2.3 lists the research gaps these works leave open, and Section 2.4 states the problem addressed by the thesis and the three research questions.

2.1 Background

Steiner trees. A multicast tree carries one copy of a stream from a source to many receivers, duplicating packets only where paths split. Finding the minimum-cost such tree is the Steiner tree problem in graphs, which is NP-hard, so practical systems use polynomial heuristics. The two standard references are **MPH** [5], which grows the tree from the source by repeatedly attaching the nearest unconnected terminal through its shortest path, and **KMB** [6], which builds a minimum spanning tree on the terminals' distance graph, expands it into shortest paths and prunes it. Both guarantee a single-rate tree at most $2(1 - 1/l)$ times optimal for l leaves.

Multi-rate trees. When receivers request different qualities, a link must carry the highest rate needed below it. Building a Steiner tree for the highest layer first and extending it layer by layer (the "top-down" heuristic) is within $\frac{\ell+1}{2}\rho$ of optimal for ℓ nested layers, where ρ is the ratio of the Steiner heuristic used [7]; dedicated algorithms for arbitrary rates reach $e \cdot \rho$ [8]. The layered MPH used in this thesis is the top-down heuristic with MPH as the Steiner step.

Multi-rate delivery. Receiver-driven layered multicast [9] sends cumulative video layers on separate multicast groups, so each receiver subscribes up to its own rate and a shared link carries the highest layer needed below it. The thesis adopts this delivery model.

SDN. Software-defined networking separates the control plane from the switches: a central controller with a global view computes trees and installs per-switch OpenFlow rules. This makes centralised, learned and repairable multicast routing practical, but every tree change costs rule updates at the switches.

2.2 Key Related Research

Wang *et al.* [1] (the base paper) formulate multi-quality multicast as a constrained minimum-flow problem and prove that the optimal flow-carrying links form a source-rooted tree in which a shared link carries the maximum demand of the receivers below it

(Lemma 1, Figure 2.1). Their graph policy network (GPN, Figure 2.2) combines a GAT encoder (hidden size 128, 4 heads), an LSTM path-history aggregator and an attention pointer decoder, with under 3 M parameters, trained by policy gradient on 30-node Erdős–Rényi graphs with 9 users. Users are routed in descending demand order, and each path grows hop by hop until it touches the existing tree, so branches are reused (Figure 2.3). On 30–50-node random graphs GPN beats shortest-path and plain-GAT routing, matches bee-colony optimization and stays slightly above GA and the DP optimum while running orders of magnitude faster (Figure 2.4). Its limitation is that the tree is built once for a fixed audience: the only dynamic test adds 1–3 users, and leaves, quality changes, link failures, capacity and rule-change cost are not modelled.

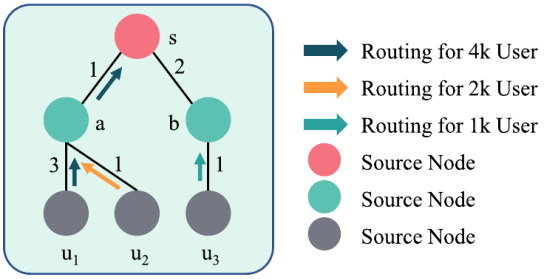


Figure 2.1: A shared link carries the highest downstream demand (Lemma 1) [1].

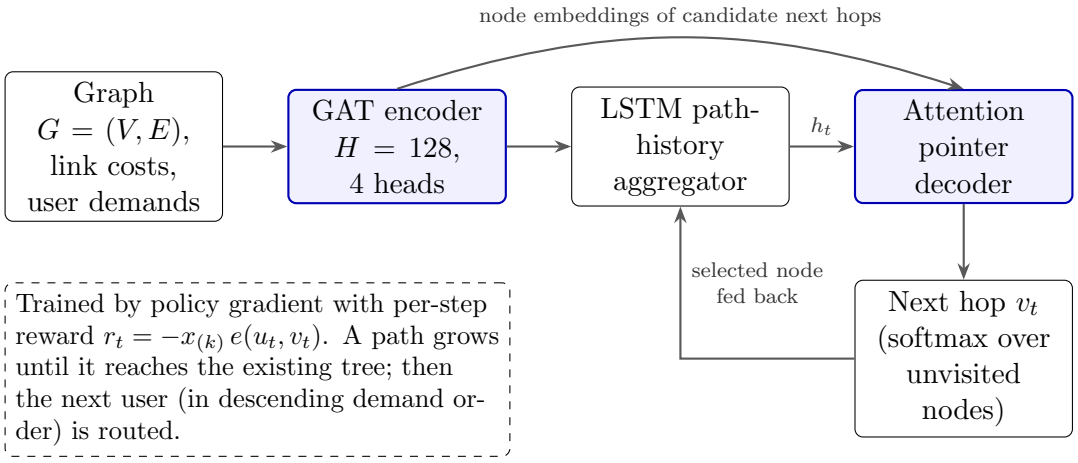


Figure 2.2: Architecture of the base paper's GPN [1].

Zhao *et al.* [10] proposed DRL-M4MR, which treats the network as four $n \times n$ matrices (tree state, residual bandwidth, delay, loss) and uses a double dueling DQN with priori-

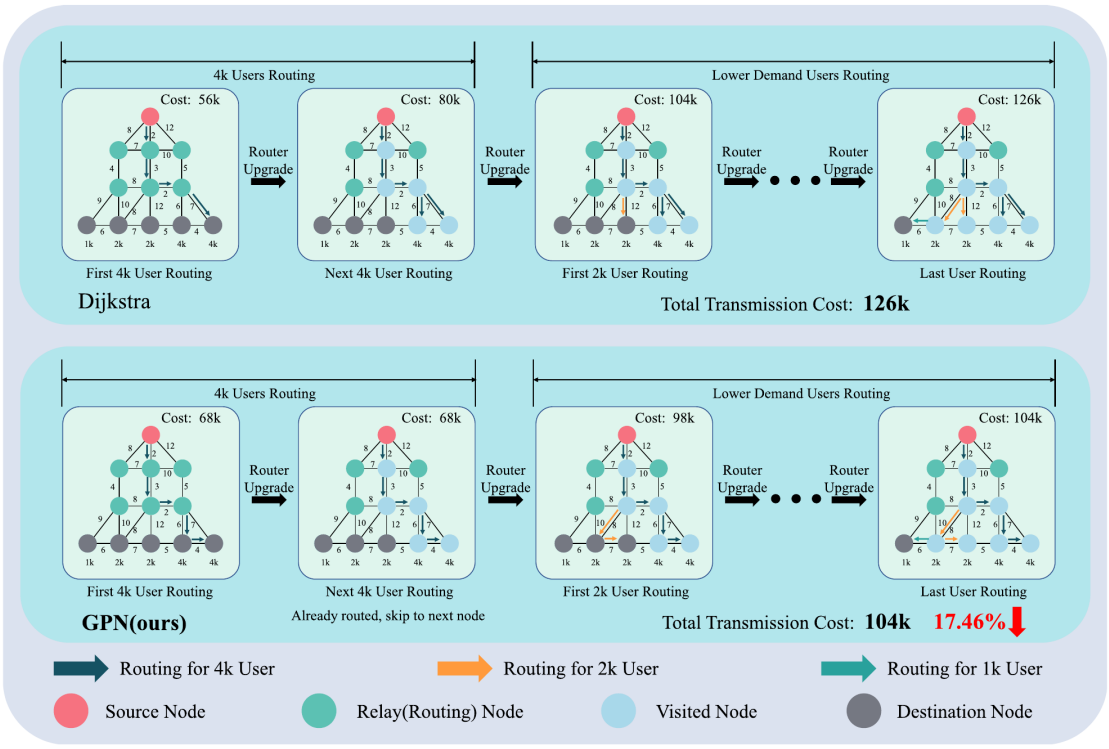
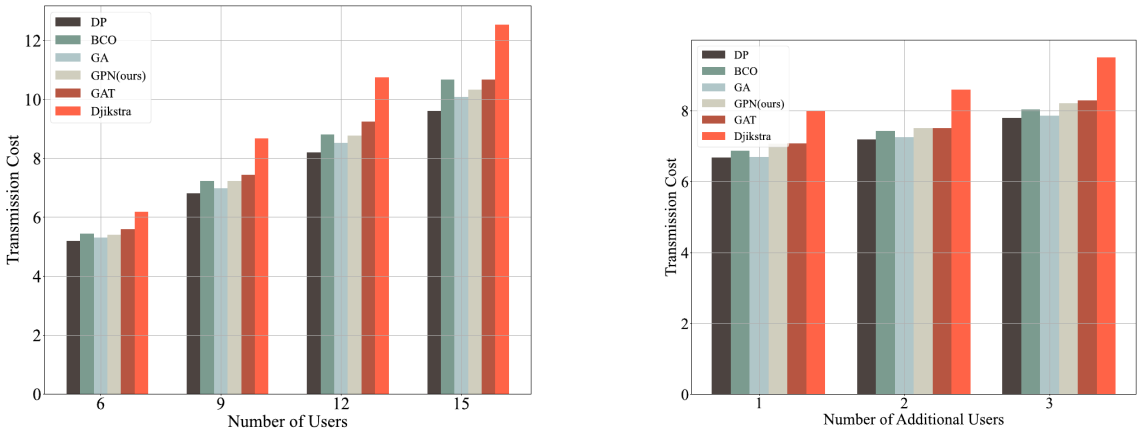


Figure 2.3: Routing under sequential user arrivals: Dijkstra (top) builds independent paths, while GPN (bottom) reuses branches and cuts total cost by 17.46% in this example [1].



(a) Cost vs. users (6–15), 50 nodes

(b) Cost after adding 1–3 users to 9

Figure 2.4: Base paper results on 50-node graphs: (a) routing cost vs. number of users; (b) its only dynamic test [1].

tised replay to add one link per step. In Ryu/Mininet on a 14-node topology it raised bottleneck bandwidth by 19.37% over KMB. Its limitation is that its trees used 15–30% more links, its fixed-size matrix input ties the model to one topology, and it re-solves for

a fixed destination set.

Zhang *et al.* [11] proposed GHAC-MR, which uses graph message passing with a PPO actor–critic and Bernoulli per-link forwarding actions, and reports lower delay and loss than KMB, GA, DRL-M4MR and a hierarchical-RL router on three topologies. Its limitation is that it also rebuilds trees for a fixed destination set without modelling leaves, quality changes or rule churn, and its own results show the hierarchical-RL router with higher bandwidth on Fat-Tree (16.74 vs. 14.63).

Imase and Waxman [12] studied the dynamic Steiner tree problem and showed that a greedy online tree is $O(\log n)$-competitive as receivers join, and that a limited number of rearrangements restores a constant-factor bound when receivers also leave. This is the cost-versus-rearrangement trade-off that this thesis measures when comparing local repair with full re-planning. Its limitation is that the analysis assumes a single rate and no link capacities, and gives no learned or practical policy for when to rearrange.

Shen *et al.* [13] proposed the Recover-aware Steiner Tree (RST) for SDN multicast, which jointly minimises tree cost and the cost of recovering from link failures, and validated it on an SDN testbed. Its limitation is that it handles a single rate and a fixed group, and does not consider receiver churn or the cost of reconfiguring switch rules.

Oostenbrink *et al.* [14] pre-install backup paths as OpenFlow fast-failover groups, so switches reroute multicast traffic locally without the controller and recover in under 10 ms. Its limitation is the extra switch memory for backup state, and the trees it protects are classical single-rate trees rather than learned multi-rate ones.

Reitblatt *et al.* [15] introduced two-phase, versioned rule updates that guarantee each packet is processed by either the old or the new configuration, never a mix. Its limitation, for this thesis, is that it makes a change safe but not cheap: it does not decide which tree to install or how many rules to change.

Table 2.1 compares these works with this thesis.

Table 2.1: Comparison of the related works

Work	Tree builder	Multi-rate	Leave / quality change	Link failure	Rule-change cost	Validation
MPH / KMB [5, 6]	Heuristic, $2(1-1/l)$ bound	No	Full re-plan	Full re-plan	Not measured	Analysis
Online Steiner [12]	Greedy + rearrangement	No	Yes (theory)	No	Edge changes bounded	Analysis
DRL-M4MR [10]	Double dueling DQN	No	No	No	No	Ryu + Mininet, 14 nodes
GHAC-MR [11]	GNN + PPO	No	No	No	No	Ryu + Mininet, 3 topologies
RST [13]	Approximation	No	No	Yes (recovery cost)	No	SDN testbed
Fast-failover [14]	Precomputed backups	No	Join/leave groups	Yes, <10 ms	Switch memory	Emulation
Base paper [1]	GAT + LSTM pointer	Yes	Adds only	No	No	Simulation, ≤50 nodes
This thesis	Base GPN + repair operators + MPH net	Yes	Yes	Yes, fast-failover	Yes ($\mu\times$ changes)	Simulation, Mininet/Ryu, 5G SA lab

2.3 Research Gaps

Learned multi-rate multicast has been shown to build good trees, but not to keep them cheap, stable and protected while the audience and the network change, and not at real receivers. The specific gaps left by the base paper are:

- **Viewers leaving or lowering quality** are not modelled; only 1–3 additions are tested, so unused branches keep consuming bandwidth (addressed by the cost model and the repair operators).
- **Late high-quality joins** are under-priced, because the reward charges only newly added links and ignores upstream upgrades (addressed by the cost model).

- **Link failures and mobility** are never tested; the only recovery is a full re-plan (addressed by the repair operators and fast-failover protection).
- **Link capacity** is not constrained, so congestion is invisible to the model (addressed by the capacity-constrained formulation).
- **Reconfiguration cost** is ignored: there is no control plane, and a re-plan re-routes every user, so rule churn and video disruption are unmeasured (addressed by the rule-change penalty and two-phase updates).
- **No quality guarantee**: GPN is 6–7% above optimum and was never compared with Steiner heuristics; our replication finds layered MPH at least as cheap on about 80% of instances (addressed by the MPH safety net of the learned policy).
- **Evaluation** uses random graphs of at most 50 nodes, and "delay" means computation time, so there is no evidence at real switches or users (addressed by the three-tier evaluation).

2.4 Problem Formulation

The problem addressed by this thesis is:

- Given an SDN transport network with link capacities, a video source with cumulative quality layers, and a stream of events (join, leave, quality change, link or switch failure), maintain a multi-rate multicast tree that minimises tree cost plus a penalty on rule changes, while keeping video disruption at receivers low.

The work answers three research questions:

- Under realistic join, leave, quality-change and failure patterns, how many rule changes and how much recomputation do local repair operators save versus full re-planning, and at what tree-cost penalty?

- Can the base paper's GNN, extended to choose keep / local repair / full re-plan and paired with a certified MPH fallback, beat fixed-threshold policies on combined cost and churn?
- Do pre-installed backup paths measurably reduce video disruption at real 5G UEs, and how much extra switch state do they cost?

3

Proposed Methodology

This chapter describes the proposed event-driven tree-maintenance method and its control loop. Section 3.1 formulates the problem, with the tree cost model, the rule-change penalty and correct upgrade cost accounting. Section 3.2 defines the local repair operators. Section 3.3 presents the learned keep/repair/re-plan policy and the capacity-aware layered MPH safety net that bounds its cost. Section 3.4 covers fast-failover backup branches and two-phase rule updates, and Section 3.5 describes the three-tier evaluation in simulation, emulation and the 5G lab.

The method keeps the base paper's GNN as the tree builder and wraps it in an event-driven maintenance loop (Figure 3.1). Each event is handled by keeping the tree, repairing it locally or re-planning it; the installed tree never costs more than $(1+\varepsilon)$ times a capacity-aware layered MPH tree; and changes reach the switches through protected, two-phase updates.

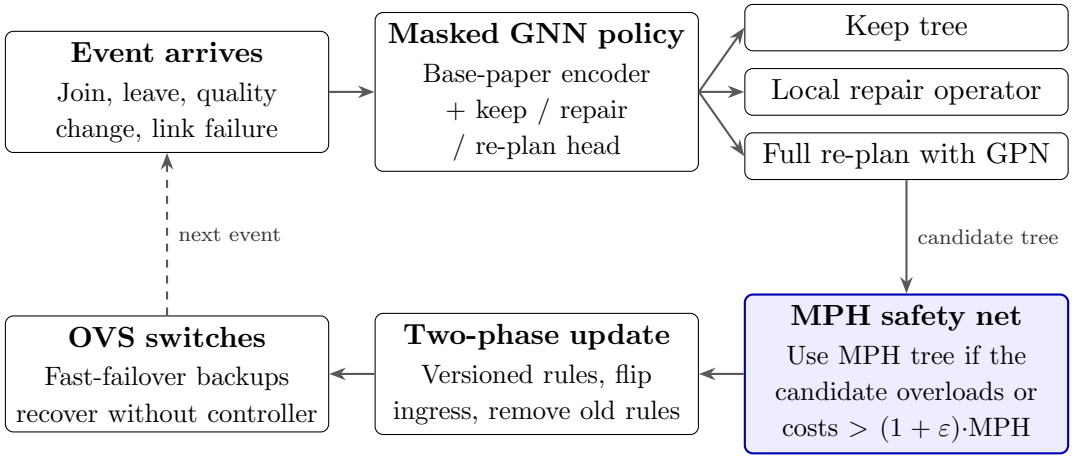


Figure 3.1: Event-driven tree maintenance loop: the learned policy chooses how much to change, the MPH check bounds the cost, and switches handle failures locally until the controller's repair arrives.

3.1 Problem Formulation

The network is a graph $G = (V, E)$ with unit cost e_{ij} and capacity c_{ij} per link. A session has a source s and receivers k with cumulative quality demand $x_k \in \{0.25, 0.5, 1.0\}$, as in [1]. By Lemma 1 of [1], a tree link carries the highest demand below it, so the cost of a tree T is

$$C(T) = \sum_{(i,j)\in T} e_{ij}\, f_{ij}, \qquad f_{ij} = \max_{k\in\mathcal{D}_T(j)} x_k, \qquad f_{ij} \le c_{ij}, \tag{3.1}$$

where $\mathcal{D}_T(j)$ is the set of receivers downstream of j. After each event t the controller chooses a new tree T_t that minimises tree cost plus a penalty on rule changes:

$$\min_{T_t}\; C(T_t) + \mu \left| R(T_t) \triangle R(T_{t-1}) \right|, \tag{3.2}$$

where $R(T)$ is the set of OpenFlow rules that realise T and μ prices one rule change; sweeping μ traces the cost-versus-churn curve used to compare local repair with full re-planning. The base paper's reward charges only new links. When a receiver with demand x_k grafts onto the tree at node a, the true marginal cost also includes raising every upstream link on the path $P(s, a)$:

$$\Delta C = \sum_{(i,j)\in \text{new}} e_{ij}\, x_k \; + \sum_{(i,j)\in P(s,a)} e_{ij} \left(\max(f_{ij}, x_k) - f_{ij}\right). \tag{3.3}$$

A leave or downgrade runs the same rule in reverse: f is recomputed upward from the leaf, and links with no receivers below are pruned.

3.2 Repair Operators

Table 3.1 lists the operators, which run inside a Ryu controller module that tracks the current tree, link loads and installed rules.

Table 3.1: Repair operators

Operator	Triggered by	Action	Rules touched
Graft	Join	Attach the receiver at the tree node with least marginal cost ΔC (Eq. 3.3)	New branch + upstream upgrades
Prune-up	Leave	Remove the leaf branch up to the first node that still serves a receiver	Removed branch only
Layer upgrade / downgrade	Quality change	Re-propagate f along the path to the source	Group memberships on that path
Re-attach	Link or switch failure	Reconnect each orphaned subtree by its cheapest path	Orphaned subtrees' new paths
Full re-plan	Policy choice or cost drift	Rebuild with GPN, then apply the MPH check	All changed rules

3.3 Learned Decision Policy and MPH Safety Net

The base GPN encoder (Figure 2.2) is reused with extra node features (in-tree flag, current f on the parent link, residual capacity, event type), and a small masked head chooses keep, local repair or full re-plan; for example, keep is masked after a failure that disconnects a

receiver. The head is trained by policy gradient on simulated event traces with reward equal to the negative of Eq. (3.2). Baselines for the learned policy are fixed-threshold rules (always repair; re-plan when cost drifts more than $\tau\%$ above MPH; re-plan every N events), all re-planning with MPH, and the main baseline for the repair operators is a full MPH re-plan per event. Whatever the policy chooses, the resulting tree passes the MPH check of Algorithm 1 before it is installed.

Algorithm 1 Capacity-aware layered MPH and install rule (one session)

for layer $x \in \{1.0,\ 0.5,\ 0.25\}$ **do** ▷ highest rate first
    $G_x \leftarrow$ links with residual + own_load $\geq x$ ▷ own_load: this session's current use
    while some layer-x receiver r is unconnected **do**
        attach the r whose cheapest path to the current tree in G_x is cheapest
          (tree links cost 0, new links cost e_{ij})
    end while
    if r cannot reach the tree in G_x **then**
        retry r at the next lower layer; drop it only if none works
    end if
end for
if candidate does not fit the residual capacities **then**
    install the MPH tree
else if $C(\text{candidate}) > (1+\varepsilon)\, C(\text{MPH tree})$ **then**
    install the MPH tree
else
    install the candidate
end if

Adding layers from the highest rate down means a later, lower layer never raises the load on a link already in the tree, so the per-layer pruning is exact: for one session, MPH finds a tree that fits whenever one exists, and a receiver it must downgrade cannot be served at its rate by any tree. The tolerance ε stops every MPH win from becoming a full re-plan, the churn that μ penalises. With no binding capacity, the installed tree costs at most $(1+\varepsilon)$ times the layered MPH tree, which is within $\frac{\ell+1}{2}\rho$ of optimal [7], about $4\times$ for three layers. That proof prices a link by the number of layers crossing it, so the bound must be re-derived for the unevenly spaced rates 0.25/0.5/1.0, or layered MPH replaced by the $e \cdot \rho$ algorithm of [8]. Whether the bound holds when capacity binds is not yet proven, and several sessions sharing links are not covered. The net's value lies in capping

the drift of repeated local repairs, which as greedy online updates can drift to $O(\log n)$ times optimal [12].

3.4 Proactive Protection and Safe Updates

For each tree link, a backup path to the downstream subtree is installed as an OpenFlow 1.3 fast-failover group [14], so the switch fails over without the controller; backups share tree links where possible to limit extra state. New rules are installed under a new version tag, the ingress is flipped to that tag, and old rules are removed after a timeout [15], so each receiver sees either the old or the new tree. Recovery time, packets lost during failover and extra flow and group entries per switch are measured.

3.5 Three-Tier Evaluation

The method is evaluated in three tiers (Table 3.2). Because the lab's Release 15/16 system has no 5G MBS [3], the tree ends at the MEC replicator and the last hop is one unicast PDU session per UE (Figure 3.2); extra edge sites are emulated with Open5GS and UERANSIM, and the lab's core and RAN settings are never changed.

Table 3.2: Three-tier evaluation plan

Tier	Setup	Scale	Measures
1 Simulation	Event-driven Python/NetworkX simulator; Topology Zoo, Rocketfuel and the base paper's random graphs	50–500 nodes, thousands of events	Cost vs. churn, policy training, scalability
2 Emulation	Mininet + Open vSwitch + the Ryu module; UDP flows at layer rates	Tens of switches	Rule churn, recovery time, switch state
3 5G lab	Emulated transport attached through the lab switch (dedicated VLAN) to the 5G SA core; MEC hosts the edge replicator	Lab CPEs and XR headsets	Stalls, startup delay, frame loss at real UEs

Workloads use Poisson viewer arrivals, exponential viewing times and Zipf audience sizes, with demand split one-third high, medium and low as in [1], plus random and correlated link and switch failures. Metrics are tree cost (and gap to an exact ILP up to about 50

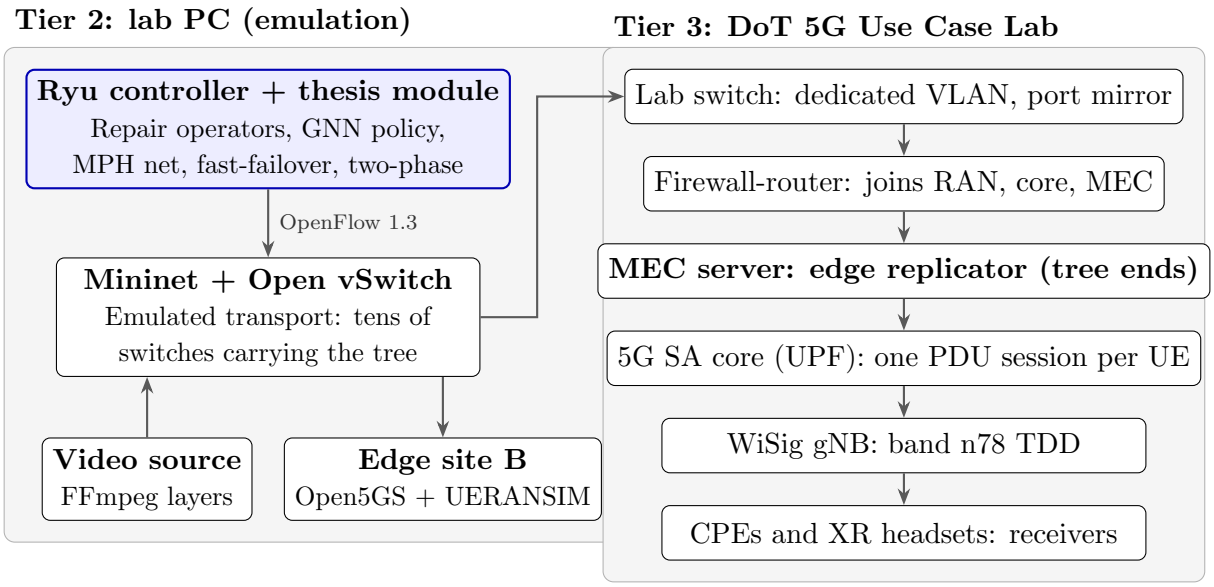


Figure 3.2: Tier 2–3 testbed: the multicast tree runs in Open vSwitch up to the MEC replicator; the 5G system carries only the unicast last hop. The thesis builds only the highlighted module.

nodes), rule changes per event, decision time, recovery time, extra switch entries and, at Tier 3, stall count, stall duration and startup delay. All tools are open source: PyTorch Geometric with the base paper's code, OR-Tools/SCIP, Ryu, Mininet, Open vSwitch, Open5GS, UERANSIM and FFmpeg.

4

Results and Discussion

This chapter reports the experiments carried out so far. Section 4.1 describes the replication setup: the public code, the fixes applied, the datasets and the hardware. Section 4.2 presents the training curves and the reproduced cost ratios against the paper's figures. Section 4.3 compares GPN with classical Steiner heuristics in cost and run time. Section 4.4 discusses what these findings mean for the thesis, Section 4.5 draws the conclusions of the mid-term work, and Section 4.6 sets out the future scope.

4.1 Replication Setup

The base paper was replicated from its public code (Figure 4.1). The paper's model, generator and reward were left untouched; every change sits in device handling, training speed or evaluation (Table 4.1).

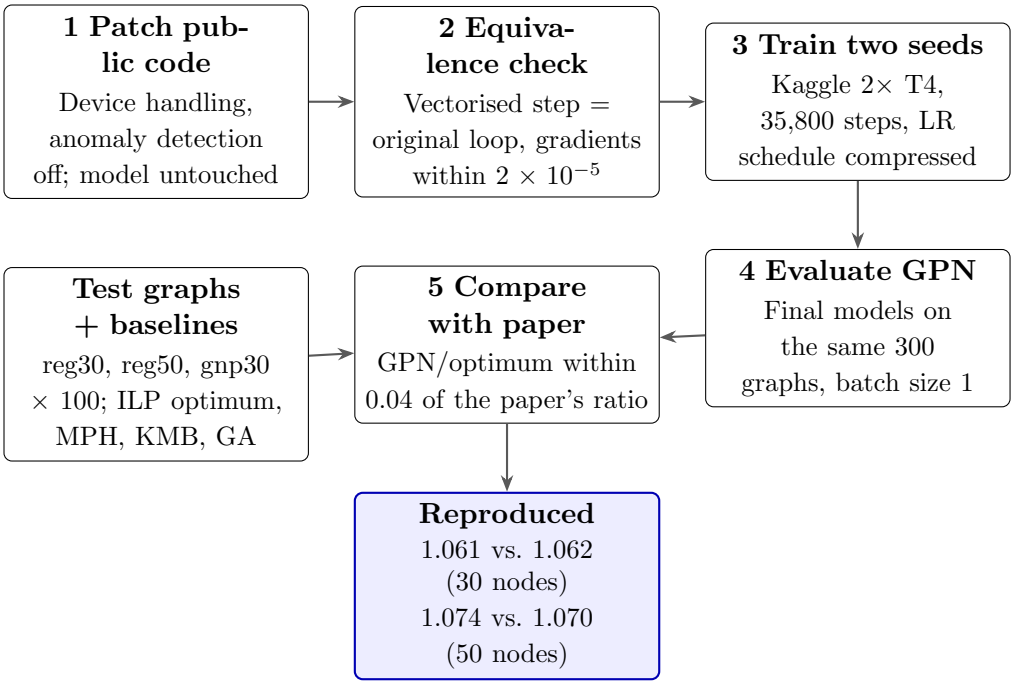


Figure 4.1: Replication pipeline: the public code reproduces the paper once patched and checked.

The public code needed several fixes: it has no baseline implementations; it hard-codes `.cuda(2)` and absolute paths; anomaly detection is on globally, slowing training several times; its validation stop check uses the training batch's features; the paper says "degree 4" while the test script uses ER $p = 0.1$ (both were evaluated, and 4-regular graphs match the paper's optimum, 6.787 vs. 6.781); and the original training loop was too slow for the free GPU quota, so a vectorised step and a compressed LR schedule were used.

4.2 Training and Reproduction

Validation cost levelled off by about step 10,000 in both seeds (Figure 4.2), and no step was skipped for a non-finite loss. The final models are reported, since the best checkpoints

Table 4.1: Replication setup compared with the base paper

Item	Paper [1]	This replication
Code	Public repo (UNIC-Lab/GNN-Routing)	Same repo, commit `ce64a61`; device handling patched, anomaly detection off
Training	29 nodes + hub, 9 users, ER $p = 0.10$; GAT ($H = 128$, 4 heads) + LSTM + pointer; Adam 5×10^{-4}; 50,000 steps, batch 16	Same graphs and model; 35,800 steps (72%) with the LR schedule compressed so the final LR factor matches (0.0169); two seeds
Hardware	Xeon 4214R + RTX 3090	Kaggle, 2× T4, 7.6 h per seed; vectorised training step checked equal to the original (gradients within 2×10^{-5}), 11–20× faster
Test graphs	30 and 50 nodes, degree 4, 12 users, 100 graphs	4-regular 30 and 50 nodes (reg30, reg50) and the repo's ER $p = 0.1$ at 30 nodes (gnp30); 12 users; 100 graphs each
Optimum	Dynamic programming	Layered directed Steiner ILP (HiGHS); all 300 instances proven optimal
Baselines	Dijkstra, GA, BCO, DP, GAT	Dijkstra, GA, layered MPH, KMB (own implementations)
Pass criterion	–	GPN/optimum within 0.04 of the paper's ratio, fixed in advance

were selected on validation data.



Figure 4.2: Validation cost on 64 fixed graphs, every 250 steps, levels off by about step 10,000 in both seeds (run length 35,800 steps; paper: 50,000).

The paper's result is reproduced (Table 4.2): GPN costs 1.061× optimal on 30-node graphs (paper: 1.062) and 1.074× on 50-node graphs (paper: 1.070), well inside the 0.04 criterion, and our exact optima agree with the paper's within their confidence intervals.

Table 4.2: Reproduction of the base paper's headline numbers

Setting	Optimum (ours, 95% CI)	Optimum (paper)	GPN (ours)	GPN (paper)	GPN / optimum (ours)	GPN / optimum (paper)	Verdict
reg30	6.806 ± 0.117	6.781	7.213	7.200	1.061	1.062	Reproduced
reg50	8.333 ± 0.138	8.207	8.949	8.781	1.074	1.070	Reproduced
gnp30	7.049	–	7.448	–	1.058	–	Repo's test generator

Costs are means over 100 graphs, averaged over two seeds; the per-seed ratios (1.0596–1.0760) differ by at most 0.004, so the result does not hinge on one seed.

4.3 Comparison with Classical Baselines

The paper does not compare GPN against Steiner heuristics. Layered MPH is the cheapest heuristic in every setting, and GPN is the most expensive method apart from Dijkstra (Table 4.3). On 50-node graphs MPH is closest to the optimum and about 45× faster than GPN (Figure 4.3); at 30 nodes it is about 65× faster.

Table 4.3: Mean tree cost over 100 graphs per setting

Setting	Optimal	MPH	KMB	GA	Dijkstra	GPN
reg30	6.806	6.972	7.071	7.176	8.700	7.213
reg50	8.333	8.582	8.737	8.815	10.545	8.949
gnp30	7.049	7.209	7.297	7.371	8.595	7.448

Per tree, GPN beats MPH on only about one instance in five (Figure 4.4). On exact (float64) tree costs, GPN is strictly cheaper on 15–23% of trees and ties on 7–18%, so MPH is at least as cheap on 77–85%. When GPN wins it wins by 2.5–3.6% of MPH's cost; when it loses it loses by 5.5–7.7%. Our GA (5.4% above optimal at 30 nodes) is weaker than the paper's undocumented GA (0.8%), so it is not presented as the paper's. Installing the cheaper of the GPN and MPH trees (the safety net with $\varepsilon = 0$ and no

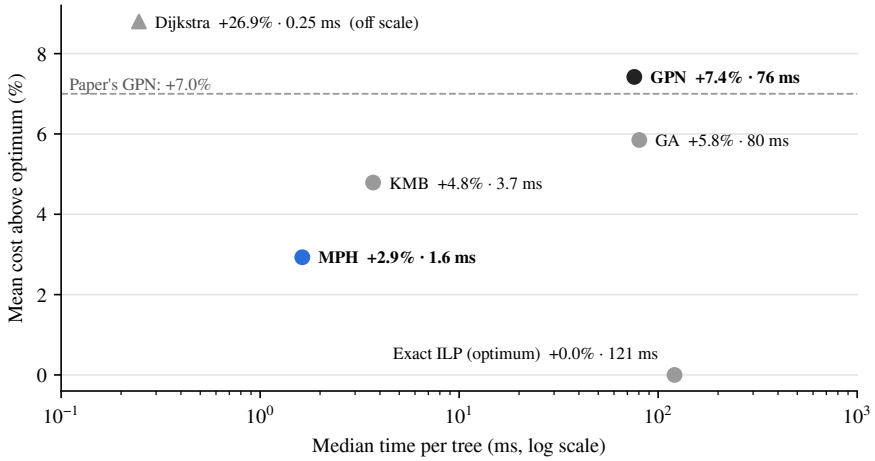


Figure 4.3: MPH builds cheaper trees than GPN and is about 45× faster (4-regular 50-node graphs, 100 instances; GPN on a T4 GPU at batch size 1, others on CPU). The dashed line marks the paper's reported GPN gap.

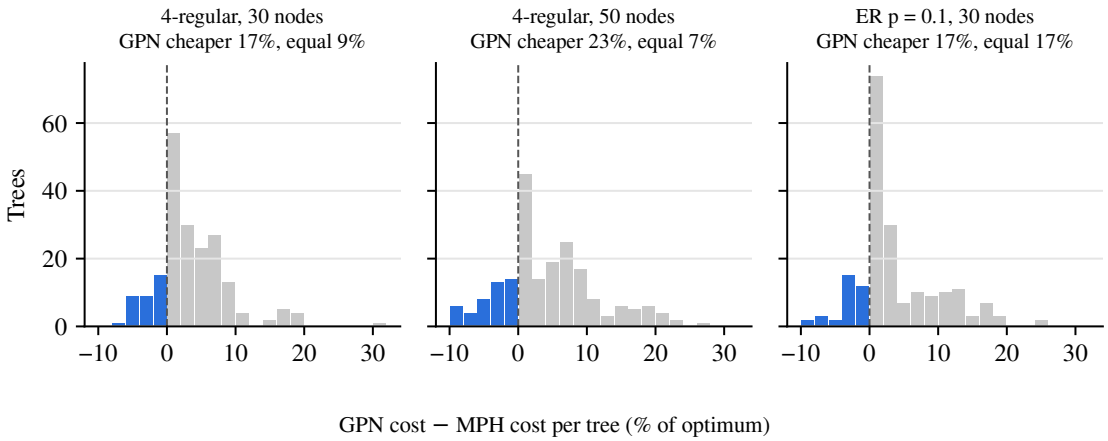


Figure 4.4: Per-tree GPN cost minus MPH cost, as a percentage of the optimum (2 seeds × 100 graphs per setting). Blue: GPN cheaper; grey: MPH cheaper or equal.

capacities) lands within 1.6–2.1% of optimal (Table 4.4), but on static trees it improves on MPH alone by under 1%.

4.4 Discussion

The public code, once patched, reproduces the paper within 0.004 and is a sound starting point for the encoder of the learned decision policy. The paper, however, misses a strong classical baseline: layered MPH is strictly cheaper than GPN on 66–75% of instances and

Table 4.4: Effect of installing the cheaper of the GPN and MPH trees ($\varepsilon = 0$, no capacities; ranges span the two seeds)

Setting	GPN strictly cheaper	Tie	Combined cost / optimum	Saving vs. MPH alone	GPN margin when it wins	GPN margin when it loses
reg30	17%	8–10%	1.019	0.43–0.45%	2.5–2.7%	5.5–5.6%
reg50	22–23%	7%	1.021	0.78–0.79%	3.4–3.6%	7.1–7.7%
gnp30	15–19%	15–18%	1.016–1.017	0.48–0.57%	3.0–3.2%	6.1%

far faster. This finding, limited to static trees, has four consequences for the thesis:

(i) cheaper one-shot trees cannot be the reason to use a GNN; its value must come from the keep / repair / re-plan decision under churn, which the design in Section 3.3 already targets;

(ii) the safety net bounds cost by a constant factor from layered MPH, not the single-rate $2(1 - 1/l)$ the proposal claimed, and must show its value on repaired trees;

(iii) the main baseline for the repair operators becomes a full MPH re-plan per event, and the threshold policies compared with the learned policy re-plan with MPH;

(iv) the risk that the GNN does not beat threshold rules is higher, so the relative weight of the repair operators and fast-failover protection will be agreed with the advisor.

Only 30–50-node static graphs with 12 users were tested, GPN was timed at batch size 1 with Python overhead, and the layered MPH is our own variant. The next check re-runs the trained model on 100–200-node graphs and Topology Zoo networks, and times batched GPN inference.

4.5 Conclusions

The first phase has fixed the research questions and objectives, reviewed the literature and identified the gap, designed the method, surveyed the DoT 5G lab (no 5G MBS,

no OpenFlow in the lab switch, a shared lab), and reproduced the base paper within 0.004 of its cost ratio, together with a missing-baseline finding against layered MPH. The remaining work therefore builds the maintenance layer on a verified starting point.

4.6 Future Scope

The remaining work builds and evaluates the maintenance layer on the verified starting point:

(i) Event-driven simulator, cost model and repair operators; first cost-vs-churn results comparing local repair with full re-planning, and a conference paper draft.

(ii) Masked keep/repair/re-plan policy with the MPH safety net; ablation against threshold rules.

(iii) Ryu module with fast-failover backups and two-phase updates; Tier 2 Mininet measurements of recovery time and switch state.

(iv) Tier 3 experiments in the 5G lab with XR headsets and CPEs as receivers.

(v) Thesis writing, open-source release of the controller module and a live demo.

If the learned policy does not beat threshold rules, the thesis still stands on the operators, protection and lab measurements. Lab experiments run on a dedicated VLAN without touching core or RAN settings.

Bibliography

[1] Xiucheng Wang, Zien Wang, Nan Cheng, Wenchao Xu, Wei Quan, and Xuemin Shen. Graph neural network-based multicast routing for on-demand streaming services in 6G networks. *IEEE Transactions on Mobile Computing*, 25(5):5988–6001, May 2026. doi: 10.1109/TMC.2025.3634587. Code: `https://github.com/UNIC-Lab/GNN-Routing`.

[2] ITU-R. Framework and overall objectives of the future development of IMT for 2030 and beyond. Recommendation ITU-R M.2160-0, November 2023.

[3] 3GPP. Architectural enhancements for 5G multicast-broadcast services; stage 2. Technical Specification TS 23.247, Release 17, 2022.

[4] Department of Telecommunications, Government of India. 100 5G labs: Infrastructure to innovations, edition 1. `https://www.dot.gov.in/static/uploads/2026/05/f7816fc9d380529182b62925ef38b96b.pdf`, 2026.

[5] Hiromitsu Takahashi and Akira Matsuyama. An approximate solution for the Steiner problem in graphs. *Mathematica Japonica*, 24(6):573–577, 1980.

[6] Lawrence Kou, George Markowsky, and Leonard Berman. A fast algorithm for Steiner trees. *Acta Informatica*, 15(2):141–145, 1981.

[7] Reyan Ahmed et al. Multi-level Steiner trees. *ACM Journal of Experimental Algorithmics*, 2019. arXiv:1804.02627.

[8] Moses Charikar, Joseph Naor, and Baruch Schieber. Resource optimization in QoS multicast routing of real-time multimedia. *IEEE/ACM Transactions on Networking*, 12(2):340–348, 2004.

[9] Steven McCanne, Van Jacobson, and Martin Vetterli. Receiver-driven layered multicast. In *Proceedings of ACM SIGCOMM*, pages 117–130, 1996.

[10] Chenwei Zhao, Miao Ye, Xingsi Xue, Jianhui Lv, Qiuxiang Jiang, and Yong Wang. DRL-M4MR: An intelligent multicast routing approach based on DQN deep reinforcement learning in SDN. *Physical Communication*, 55:101919, December 2022. arXiv:2208.00383.

[11] Peiying Zhang, Lijuan Chen, Jian Wang, Yujie Yuan, Chun Sing Lai, and Lizhuang Tan. Topology-aware deep reinforcement learning for dynamic multicast routing in software-defined networks. *Future Internet*, 18:281, 2026. doi: 10.3390/fi18060281.

[12] Makoto Imase and Bernard M. Waxman. Dynamic Steiner tree problem. *SIAM Journal on Discrete Mathematics*, 4(3):369–384, 1991.

[13] Shan-Hsiang Shen, Liang-Hao Huang, De-Nian Yang, and Wen-Tsuen Chen. Reliable multicast routing for software-defined networks. In *Proceedings of IEEE INFOCOM*, 2015.

[14] Jorik Oostenbrink, Niels van Adrichem, and Fernando Kuipers. Fast failover of multicast sessions in software-defined networks. arXiv:1701.08182, 2017.

[15] Mark Reitblatt, Nate Foster, Jennifer Rexford, Cole Schlesinger, and David Walker. Abstractions for network update. In *Proceedings of ACM SIGCOMM*, pages 323–334, 2012.